

"The Scrum Master is responsible for ensuring Scrum is understood and enacted. Scrum Masters do this by ensuring that the Scrum Team adheres to Scrum theory, practices, and rules. The Scrum Master is a servant-leader for the Scrum Team. The Scrum Master helps those outside the Scrum Team understand which of their interactions with the Scrum Team are helpful and which aren't. The Scrum Master helps everyone change these interactions to maximize the value created by the Scrum Team. (...continues...)"

E.2.3 User Story

E.2.3.1 Practice

Graphical syntax

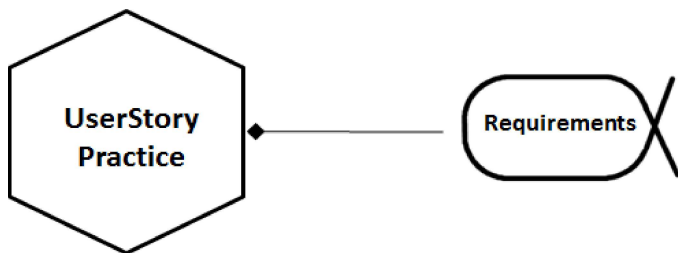


Figure E.14 – User Story practice

Textual syntax

```

kernel ESSENCE_kernel:
    "...
owns {
    alpha Requirements:
        "...."
        with states {
            state someState {
                "...
            }
        }
}

practice UserStory:
    "...
    with objective "..."
owns {
    ESSENCE_kernel.Requirements contains 1..N UserStory
    workProduct UserStoryCard:
}
  
```

E.2.3.2 Work Products

E.2.3.2.1 User Story

A User Story can be seen as a requirement item sub-alpha of Requirements that you want to monitor the state of. This requirement item is described by a User Story Card.

Graphical syntax

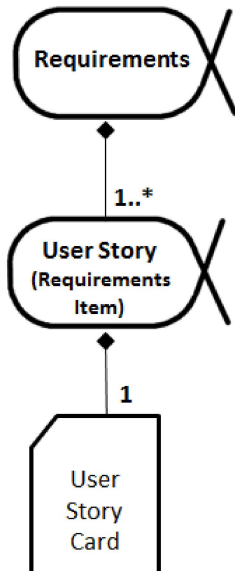


Figure E.15 – User Story

Textual syntax

alpha UserStory:

"A User Story is an Independent, Negotiable, Valuable, Estimatable, Small, Testable requirement (INVEST)"

```
with states {
  state Described {
    "The User Story is described."
    checks {
      item c1 {"User Story is described by the customer."}
      item c2 {"User Story is prioritized by the customer."}
    }
  }
  state Understood {"The User Story has been analyzed by the Team"}
  checks {
    item c1 {"The User Story has been broken down into tasks by the developers."}
    item c2 {"The User Story has been estimated by the developers."}
  }
}
state Implemented {"The User Story has been implemented."}
checks {
  item c1 {"The User Story has been implemented."}
  item c2 {"The implementation has been tested."}
}
```

```

    }
    state Fulfilled {"The User Story has been fulfilled."
        checks {
            item c1 {"The Customer has approved the implementation."}
        }
    }
}

```

workProduct UserStoryCard:

```

"The User Story Card contains the description of the User Story. User
stories generally follow the following template:
\"As a <role>, I want <goal/desire> so that <benefit>\"
\"As a <role>, I want <goal/desire>\""

with levels {
    level someLevel {
        "...\"
    }
}

```

E.2.3.3 Activities

E.2.3.3.1 Write User Story

Graphical syntax

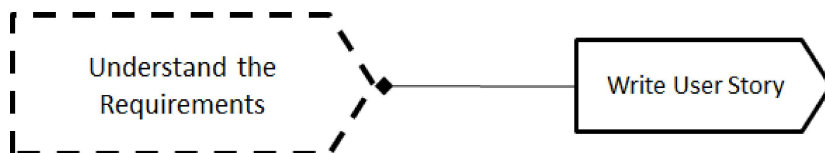


Figure E.16 – Write User Story

Textual syntax

```

activity WriteUserStory:
    "...\"
    targets UserStory.Described

    WriteUserStory -- "part-of" --> SEMAT_kernel.UnderstandTheRequirements

```

E.2.3.3.2 Prioritize User Story

Graphical syntax

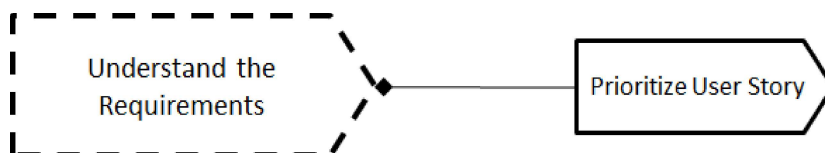


Figure E.17 – Prioritize User Story

Textual syntax

```
activity PrioritizeUserStory:
    "...
    targets UserStory.Described

    PrioritizeUserStory -- "part-of" --> SEMAT_kernel.UnderstandTheRequirements
```

E.2.3.3.3 Estimate User Story

Graphical syntax

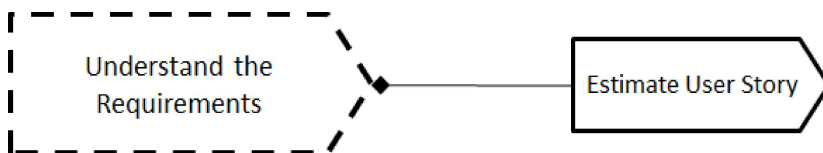


Figure E.18 – Estimate User Story

Textual syntax

```
activity EstimateUserStory:
    "...
    targets UserStory.Understood

    EstimateUserStory -- "part-of" --> SEMAT_kernel.UnderstandTheRequirements
```

E.2.4 Multi-phase Waterfall

In some practices in common use, there are multiple phases of Requirements Definition, each adding more detail.

- Multiple Requirements and Design Activities normally flow top down.
- Multi-phase Testing Activities normally flow bottom up.

This practice example is closely related to the so-called V-Model for software process engineering http://www.the-software-experts.de/e_dta-sw-process.htm.

- Actual Flow of Activities associated with each phase can be quite complex in a real project.
- Requirements alpha specializations are needed to model requirement documents from each phase.

E.2.4.1 Activities

The general form of the V-model of Activities for the Multi-phase Waterfall practice is shown in Figure E.19.

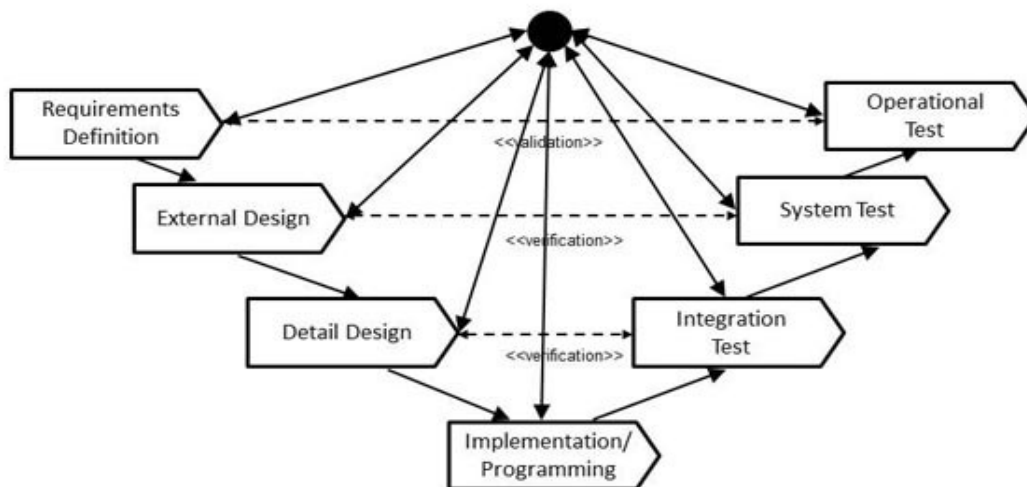


Figure E.19 – Multi-phase Waterfall Practice Activities Flow

Figure E.19 shows an example of “V-Model” for Multi-phase Waterfall Practice. Each Test Activity verifies/validates work products of one Requirements/Design Activity. Normal progression flows from left to right. If defects are detected or rewind is required, process flows back to appropriate point thru the depicted virtual node

E.2.4.1.1 Requirements Definition Phase

Description	Major work products
- Confirm the systematization requirements to define functional (system functions, data, interface) and non-functional requirements - Define and outline design of the system and examine the feasibility of the system. - Develop a project plan and establish management measurers to carry out the project.	- Use cases & Scenario - Business flows - Business rules - Data model (High-level) - Execution environment prescription (as Non-functional requirement) - Business operational test spec.

E.2.4.1.2 External Design Phase

Description	Major work products
• Design high-level specifications for end users such as system functions, data, interfaces, screens and print-form • Design the system architecture and operation measures. • Investigate the current assets (applications, system configuration, data) to determine which resources should be transferred to the new system. • Develop a total test plan.	• Application architecture spec. • Conceptual data model • Screen Design spec. • Printing-form design spec. • Process structure spec. • Interface design spec. • Message & Code design • Detail Non-functional requirements • System test specification.

E.2.4.1.3 Detailed Design Phase

Description	Major work products
• Design the system internal structure (ex. program unit, database physical structure) and interfaces between programs based on the outline specifications. • Design an operation management system, security system, and methods for transition of the current resources.	• Software component/module spec. • Physical Database schema specification • Detail screen spec.(screen constituent) • Performance design • Security design • Integration test spec

E.2.4.1.4 Implementation/Programming Phase

Description	Major work products
• Define program structure and design program logic • Develop and complete programs based on the program design • Implement the database based on the data model. • Test each program module individually to verify correctness and quality.	• Source code • Middleware/Hardware configuration specification. • Database definition Language

E.2.4.1.5 Integration Test Phase

Description	Major work products
• Test each process by integrating programs to verify the application. • Test interfaces between all processes • Confirm interfaces between external systems	• Result reports for Integration test spec.

E.2.4.1.6 System Test Phase

Description	Major work products
• Test the business system functions on the actual machines. • Test the entire system by evaluating system performance, reliability, operability, security, etc.	• Result reports for System test spec.

E.2.4.1.7 Operational Test Phase

Description	Major work products
- Test business operations in the real environment with actual machines and real data. This test is performed by end users. - Validate the business functions, performance, reliability, operability, and security. - Make decision to transit from test operation to real operation, and perform a transition.	Result reports for business operational test.

E.2.4.2 Alpha Extensions for Multi-Phase Waterfall Requirements

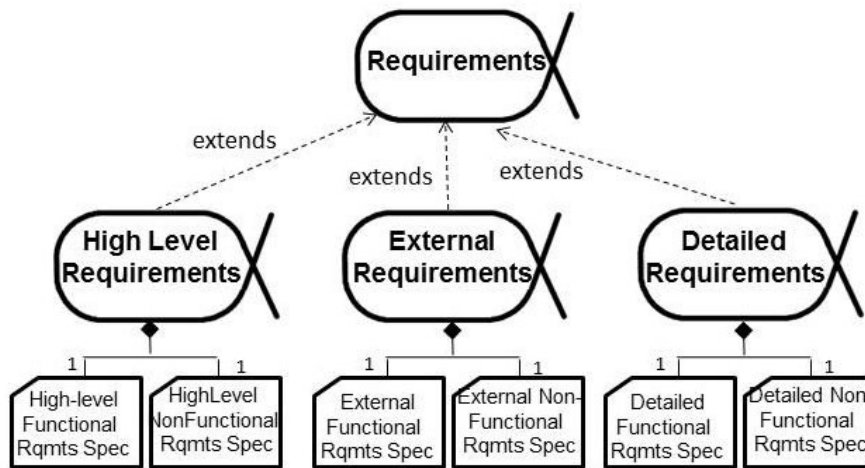


Figure E.20 – Multi-phase Waterfall Requirements Alpha Extensions and Requirements Spec Work Products

High Level Requirements Specs (Functional and Non-Functional) are produced by Requirements Definition Activity.

External Requirements Specs (Functional and Non-Functional) are produced by External Design Activity.

Detailed Requirements Specs (Functional and Non-Functional) are produced by Detailed Design Activity.

Each Requirements extension Alpha has:

- Its own state values, the same as specified for the Requirements Alpha;
 - Conceived; Bounded; Coherent; Described; Addressed; Fulfilled
- Functional and Non-Functional Requirements Spec Work Products
 - each having Sub-Alphas for every Requirement Item, with their own state values (the same as specified for the Requirement Item Sub-Alpha Kernel Extension)

- Requirements Alpha Extension state transitions conditional on Requirements Item Sub-alpha state transitions

E.2.4.3 Lifecycle Diagram for Multi-Phase Waterfall Requirements Alpha Extensions

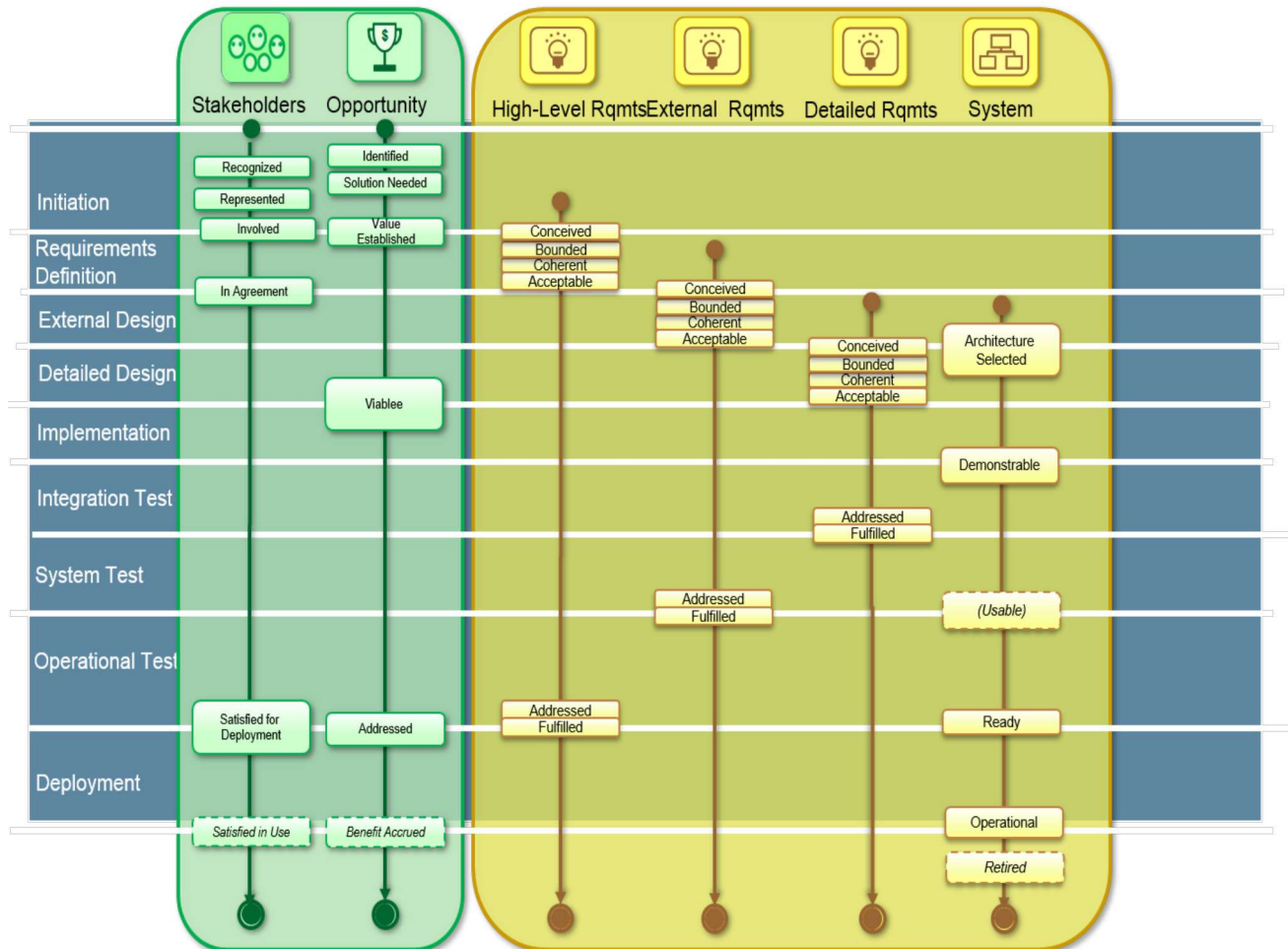


Figure E.21 – Lifecycle Diagram for Multi-Phase Waterfall Requirements Alpha Extensions

E.2.4.4 Extensions of Requirement Item Alpha for Tracking Individual Multi-Phase Waterfall Requirement Items

If a project needs to track to state of each individual requirement item, the following Sub-Alpha extensions of the Requirement Item kernel Extension Sub-alpha can be employed.

The individual Requirement Work products are part of their respective Requirements Spec (Functional or Non-Functional) associated with their parent Requirements Alpha Extension.

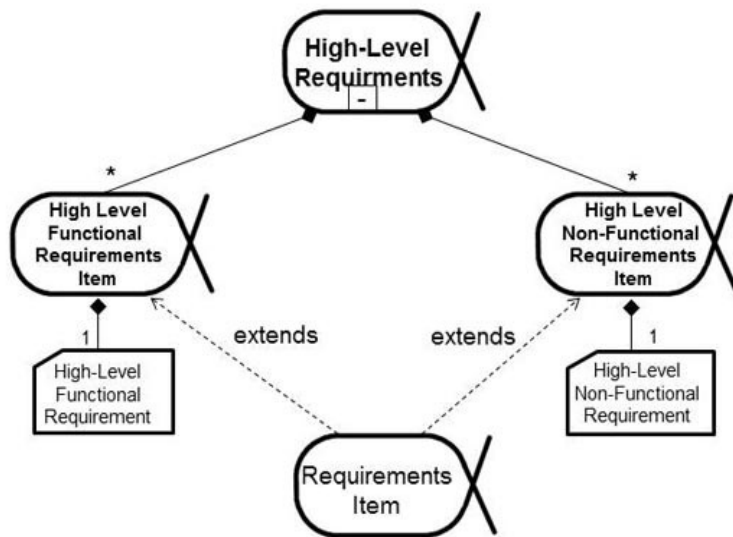


Figure E.22 – High-Level Requirements Sub-Alphas and Requirement Work Products

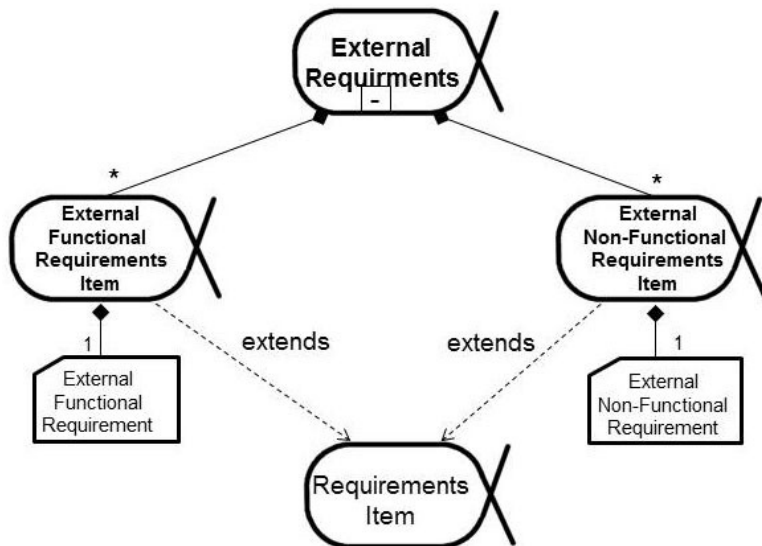


Figure E.23 – External Requirements Sub-Alphas and Requirement Work Products

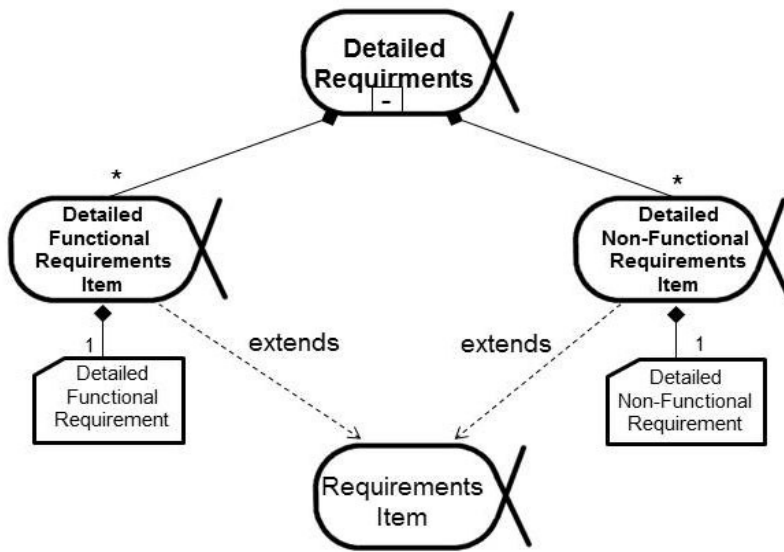


Figure E.24 – Detailed Requirements Sub-Alphas and Requirement Work Products

E.2.5 Lifecycle Examples

The Essence Kernel enables practices to define lifecycles by sequencing a number of patterns, one for each phase and/or milestone in the lifecycle.

This subclause provides illustrations of a number of typical software engineering lifecycles:

- A Unified Process lifecycle
- A waterfall lifecycle
- A set of complementary application development lifecycles
- A funding and decision making lifecycle

When reading these subclauses one should bear in mind that a lifecycle practice can do more than just arrange the alpha states, it can also add items to the checklists, activities to formally review the milestones and any other planning or review guidance it sees fit.

All the lifecycles are illustrated using the template shown in Figure E.25.

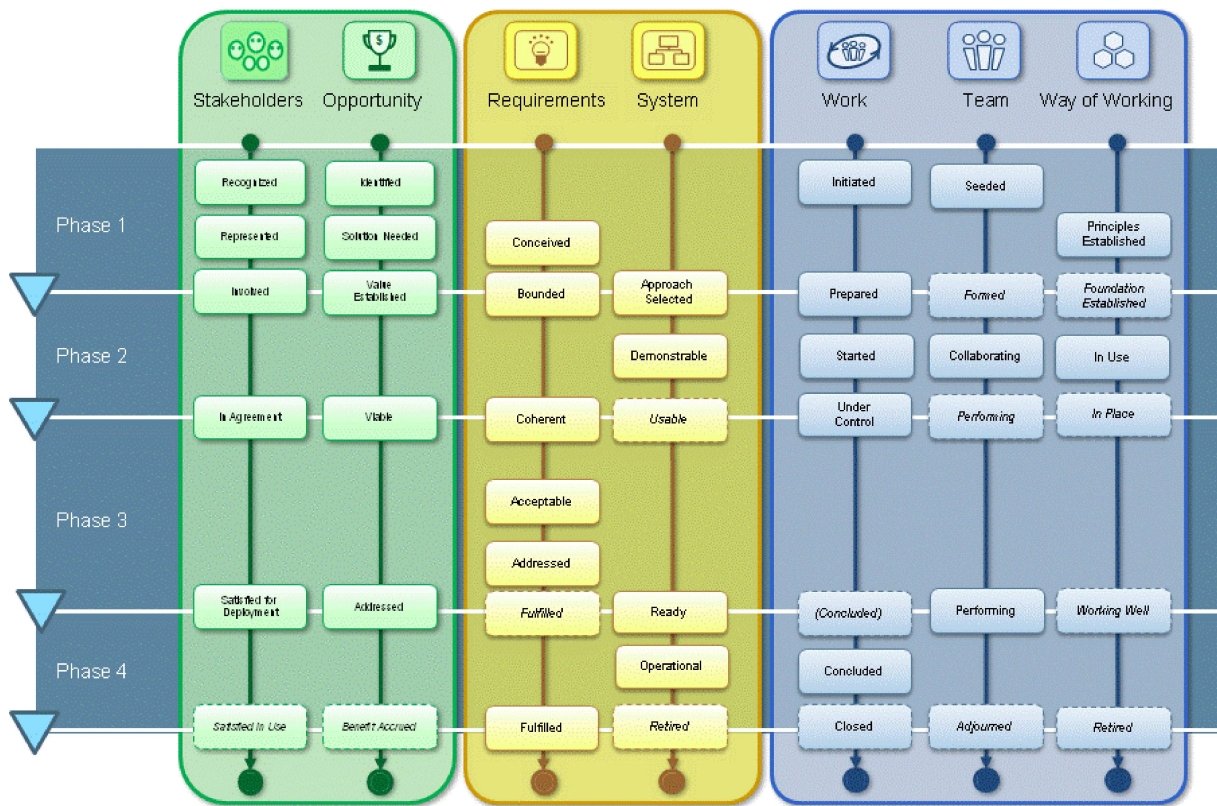


Figure E.25 – Lifecycle template

Each Kernel Alpha and its states are shown in a vertical column with their creation at the top and their destruction at the bottom. Milestones are shown as a vertical bar across the grid starting with an inverted triangle to represent the milestone and continuing with a white line over which are shown the states to be achieved to successfully pass the milestone. Where achieving a state is either recommended or optional the state is shown with a dashed outline and italicized text.

E.2.5.1 The Unified Process Lifecycle

An illustration of the Unified Process Lifecycle is shown in Figure E.26. In the Unified Process Lifecycle there are four phases: Inception, Elaboration, Construction and Transition. Each of these ends in a distinct milestone: Lifecycle Objectives Milestone, Lifecycle Architecture Milestone, Initial Operational Capability, Project End. In Figure E.26, the milestones are represented by the blue inverted triangles but the names are suppressed to keep things simple.

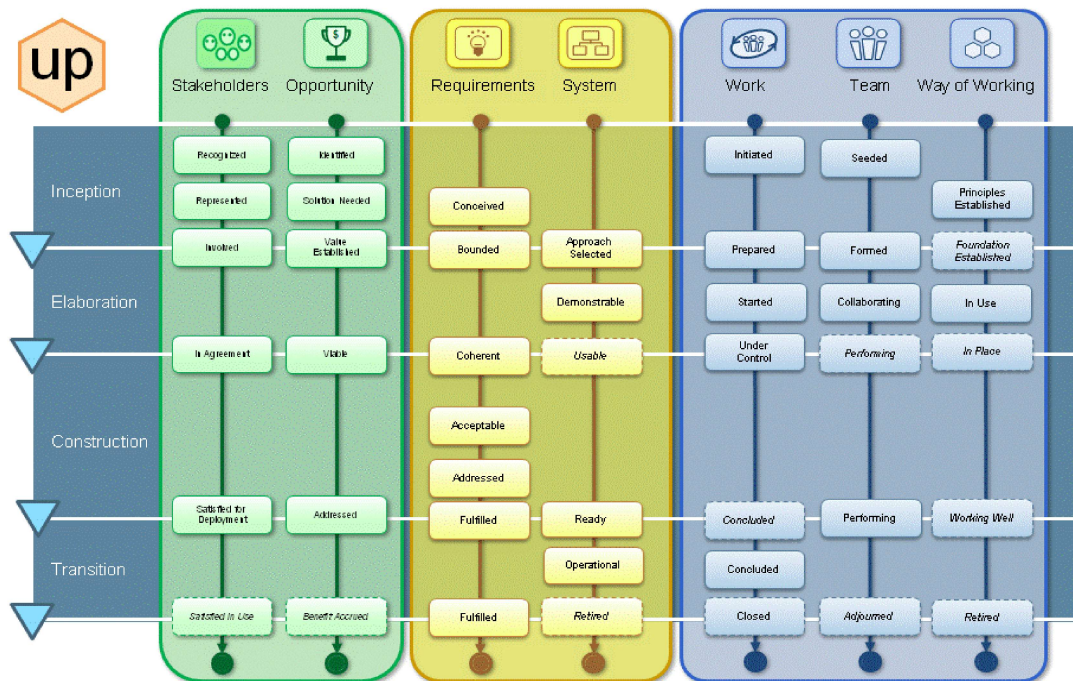


Figure E.26 – The Unified Process lifecycle

E.2.5.2 The Waterfall Lifecycle

An illustration of a Waterfall Lifecycle is shown in Figure E.27. In this case there are six phases: Initiation, Requirements, Analysis and Design, Implementation, Testing, and Deployment. Each of these ends in a distinct milestone, which in this case are not named.

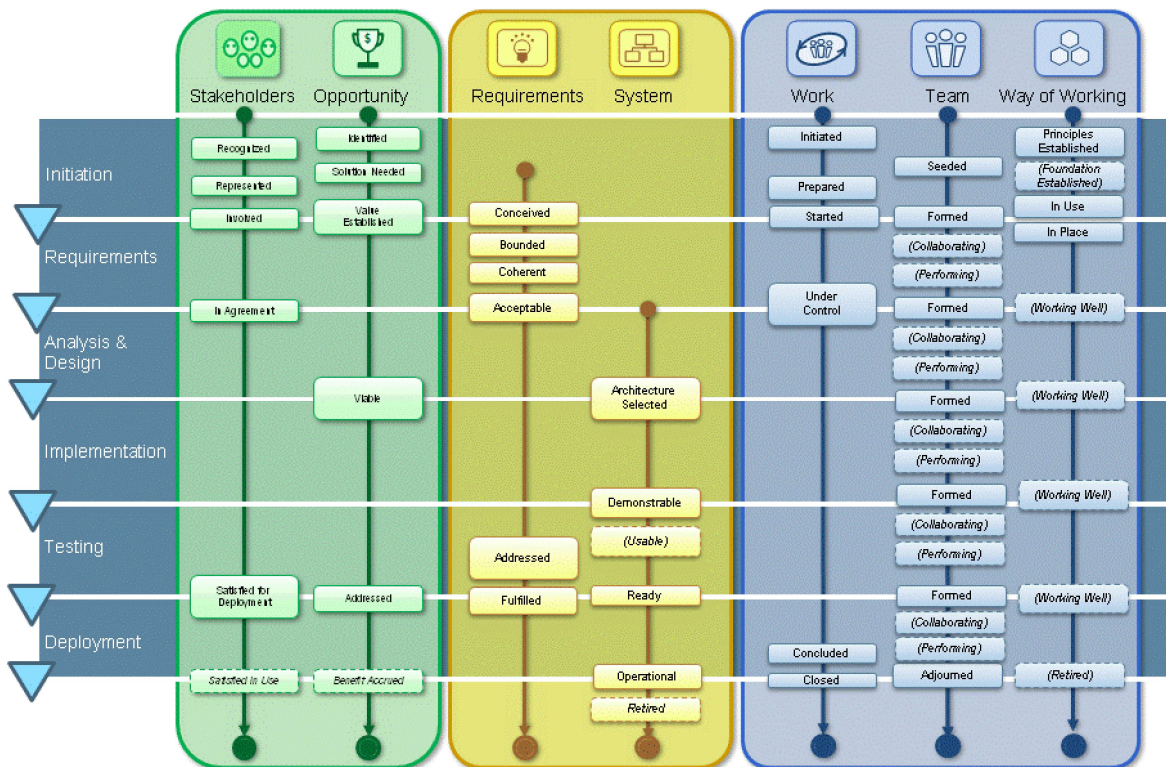


Figure E.27 – A Waterfall lifecycle

Of most interest here are:

1. The fact that there is no work on the system itself until the Analysis and Design Phase at the earliest.
2. Different team formations are used for each phase and so the state of the team keeps getting set back to *formed* with the hope that the new team will be *collaborating* and *performing* before the end of its phase.
3. The Requirements are *acceptable* by the end of the Requirements Phase and then not progressed again until the Testing Phase.

E.2.5.3 A set of complementary application development lifecycles

The Kernel can be used in much more subtle ways than in the previous two examples. It is not uncommon for application development organizations to need multiple lifecycles to cope with the different types and styles of development that they undertake. Figure E.28 shows four complementary lifecycle models illustrating the typical demands made upon an application development organization. This example is taken from a real software development organization and uses their names for the four lifecycle models.